Universidad Politécnica de Madrid
ETSI Informáticos

Inteligencia Artificial Explicable, curso 2026/27

Interpretación de modelos para la predicción de dificultades de pago

Proyecto práctico de la Unidad 2. Fase I (entrega preliminar)

Integrante	**Correo electrónico**
Iker Rosales Saiz	iker.rosales@alumnos.upm.es
Pratham Jaikishan Bakshani Karamchandani	
Hugo Aguilar Sanchez	

25 de septiembre de 2026

1. Introducción

La evaluación del riesgo crediticio requiere un equilibrio delicado entre la capacidad predictiva de los modelos y la transparencia de sus decisiones. En este proyecto se aborda el problema de predecir la dificultad de pago de clientes de tarjetas de crédito para el mes siguiente, representada por la variable binaria `payment_difficulty_next_month`.

Los datos proceden del conjunto *Default of Credit Card Clients* de UCI [12], con una variable objetivo y algunas variables semisintéticas, por lo que los resultados no deben extrapolarse a una población real. El conjunto de entrenamiento (25 000 clientes, 15 predictores) presenta un desequilibrio de clases: aproximadamente el 21,2 % de los clientes tiene dificultades de pago, frente al 78,8 % que no. Un clasificador trivial que prediga siempre la clase mayoritaria alcanza por tanto un 78,8 % de exactitud (*accuracy*), que es la línea base sobre la que medimos cualquier ganancia real.

En decisiones de alto impacto como la concesión de crédito, es deseable que cada clasificación de riesgo vaya acompañada de una justificación clara para el cliente y para los auditores [10]. Por ello, en esta primera fase nos centramos exclusivamente en **modelos lógicos interpretables** (árboles de decisión, listas de reglas y conjuntos de reglas), que permiten inspeccionar de forma explícita las condiciones que conducen a cada predicción [8].

Los objetivos de esta entrega son:

1. Realizar un análisis exploratorio y un preprocesado de los datos para identificar los factores asociados a la dificultad de pago.
2. Comparar distintos algoritmos lógicos (CART, GOSDT, Greedy Rule List, Bayesian Rule List y RIPPER) bajo una misma validación cruzada.
3. Explicar globalmente el modelo seleccionado y analizar localmente clientes concretos para comprobar la coherencia de sus reglas.

En resumen, el modelo propuesto es un árbol óptimo GOSDT de **7 hojas** con una exactitud en validación cruzada de 81,54 %, solo 0,1 puntos por debajo de un *gradient boosting* de caja negra. Casi toda la señal se concentra en el comportamiento de pago reciente: estar en retraso, el número de meses con retraso y pagar una fracción muy pequeña de la factura.

2. Método

2.1. Preprocesado y protocolo de evaluación

El conjunto de datos contiene variables sociodemográficas (edad, sexo, educación, estado civil, límite de crédito), el último estado de pago y métricas agregadas de los últimos 6 meses (retrasos, importes facturados, pagos y ratios). No hay valores nulos. Existen 18 filas duplicadas si se ignora el identificador, que se mantienen al ser coincidencias plausibles dados los importes redondeados.

Codificación. Los árboles y las reglas son invariantes a transformaciones monótonas de las variables [8], por lo que las variables numéricas se usan sin escalar ni transformar, a pesar de sus colas largas. Las cuatro variables categóricas se codifican en *one-hot* (23 variables en total). Los códigos negativos de `latest_repayment_status` no están documentados; se mantienen como valores numéricos para que los árboles puedan agruparlos mediante umbrales sin necesidad de asignarles un significado.

Variables sensibles y de asignación selectiva. La variable `sex` se mantiene como predictor, pero el análisis exploratorio muestra tasas de dificultad casi idénticas (20,8 % en mujeres y 21,8 %

en hombres) y ninguno de los modelos finales la utiliza. La auditoría descriptiva de errores por subgrupo se reserva para la Fase II, sin extraer conclusiones de equidad. Del mismo modo, `repayment_assistance_plan` se asocia a más dificultad (32,7 % frente a 18,1 %), pero el plan se asigna a clientes que ya muestran problemas, por lo que esta asociación no se interpreta como causal.

Binarización. GOSDT y la Bayesian Rule List requieren variables binarias. Siguiendo el *notebook* de clase, construimos 25 indicadores del tipo `variable` $\geq$ umbral, con umbrales tomados de los saltos observados en el análisis exploratorio (por ejemplo, ratio de pago $\geq 0{,}03$, 6 meses con retraso o edad ≥ 25). RIPPER discretiza internamente las variables numéricas por cuantiles.

Evaluación. Todas las decisiones (hiperparámetros, comparación de modelos y umbral) se toman con la misma **validación cruzada estratificada de 5 particiones** (semilla 42). Además de la exactitud, que es la métrica con la que se evaluará `test.csv`, registramos el área bajo la curva ROC (AUC), la exactitud balanceada y el F1 de la clase positiva, ya que con este desequilibrio la exactitud por sí sola es poco informativa. GOSDT solo devuelve etiquetas, así que usamos como probabilidad la tasa de positivos de la hoja en entrenamiento para poder calcular su AUC.

Criterio de selección. Para cada algoritmo elegimos **el modelo más pequeño cuya exactitud media esté a menos de** 0,002 **de la mejor de su rejilla**, lo que equivale aproximadamente a una desviación típica entre particiones. Así se prioriza la interpretabilidad cuando las diferencias de rendimiento no son significativas.

Umbral de clasificación. Con las probabilidades fuera de muestra (*out-of-fold*) evaluamos umbrales entre 0,2 y 0,8. En todos los modelos el umbral óptimo está en torno a 0,5, o 0,5 cae dentro de la meseta de máxima exactitud, así que se mantiene la regla de decisión por defecto.

2.2. Modelos lógicos

- **CART** (`DecisionTreeClassifier` de scikit-learn [2, 9]): rejilla con `max_depth` $\in \{1, \ldots, 8, 10, 12\}$ y `min_samples_leaf` $\in \{1, 50, 200\}$.
- **GOSDT** (*Generalized and Scalable Optimal Sparse Decision Trees* [5]): en lugar de construir el árbol de forma voraz, minimiza globalmente $R(d) = \ell(d) + \lambda\,|d|$, donde ℓ es el error de clasificación en entrenamiento y $|d|$ el número de hojas [8]. Se usa la versión 0.1.8 de `gosdt`, envuelta en una clase propia (`GOSDTClassifier`) compatible con la interfaz de scikit-learn. Rejilla: $\lambda =$ `regularization` $\in \{0{,}005, 0{,}002, 0{,}001, 0{,}0005\}$ y `depth_budget` $\in \{3, 4\}$, donde `depth_budget` cuenta también el nivel de las hojas. Con `depth_budget` $= 5$ cada ajuste agotaba el límite de 60 s sin mejorar el árbol, por lo que se descartó.
- **Listas de reglas** (`imodels` [11]): una **Greedy Rule List** (`max_depth` $= 6$) como primera aproximación y una **Bayesian Rule List** [4] sobre las variables binarias (reglas de hasta 2 condiciones, soporte mínimo del 5 %, longitud esperada de la lista 3, 2 cadenas MCMC de 1000 iteraciones). La BRL tarda en torno a 1.5 minutos por ajuste, por lo que no se ajustó con rejilla.
- **Conjunto de reglas**: **RIPPER** [3] (paquete `wittgenstein`), que aprende reglas para la clase positiva por cobertura secuencial. Rejilla: `max_rules` $\in \{3, 5, 8, 12\}$ y `max_rule_conds` $\in \{2, 3\}$.

Búsqueda extendida sobre GOSDT. Una vez elegido GOSDT como mejor familia, ampliamos su búsqueda a 20 configuraciones: `regularization` $\in \{0{,}002, 0{,}001, 0{,}0005, 0{,}0002, 0{,}0001\}$,

`depth_budget` $\in \{3, 4\}$ y dos binarizaciones. La primera es la *base* (25 variables) y la segunda una *fina* (34 variables), con cortes adicionales en el estado de pago, los meses con retraso, el ratio de pago (desde 0,01), el pago total y la factura media (distintos de cero) y la edad (≥ 30).

***Threshold guessing* y referencia de caja negra.** Para no depender de umbrales elegidos a mano, probamos también el *threshold guessing* propuesto por los autores de GOSDT [6]. Dentro de cada partición de entrenamiento se ajusta un *gradient boosting* con árboles de profundidad 1, se conservan los umbrales que utiliza (con selección hacia atrás) y GOSDT se entrena sobre ellos. Rejilla: `n_est` $\in \{20, 40, 80\}$ y `regularization` $\in \{0{,}001, 0{,}0005, 0{,}0002\}$, con `depth_budget` $= 4$. Por último, evaluamos un `HistGradientBoostingClassifier` con parámetros por defecto como referencia de caja negra, para cuantificar cuánta exactitud se sacrifica al usar un modelo interpretable.

3. Resultados

La Tabla 1 resume el rendimiento en validación cruzada de todos los estimadores ajustados.

Clasificador	Ajustes clave	Tamaño	Accuracy	AUC	F1
`DummyClassifier`†	*Clase más frecuente*	*1*	*0.7879 ± 0.0001*	*0.500*	*0.000*
`HistGradientBoosting`‡	*Parámetros por defecto*	–	*0.8164 ± 0.0026*	*0.768*	*0.401*
Árbol de decisión					
`DecisionTreeClassifier`	`{max_depth=4, min_samples_leaf=200}`	14	0.8133 ± 0.0023	0.747	0.349
`GOSDT`	**`{regularization=0.0005, depth_budget=4}`**	**7**	**0.8154 ± 0.0020**	**0.649**	**0.376**
`GOSDT` + *threshold guessing*	`{n_est=40, regularization=0.0005, depth_budget=4}`	7	0.8136 ± 0.0041	0.652	0.379
Lista de reglas					
`GreedyRuleListClassif.`	`{max_depth=6}`	8	0.7878 ± 0.0001	0.719	0.000
`BayesianRuleListClassif.`	`{maxcardinality=2, minsupport=0.05, listlengthprior=3}`	13	0.8053 ± 0.0055	0.744	0.394
Conjunto de reglas					
`RIPPER`	`{max_rules=8, max_rule_conds=3}`	9	0.8043 ± 0.0036	0.588	0.311

† Referencia inferior: predice siempre la clase mayoritaria (sin dificultad). ‡ Referencia de caja negra, no interpretable. En negrita, el modelo propuesto. Tamaño: número de hojas o de reglas, incluida la regla por defecto. Accuracy: media ± desviación típica entre particiones; AUC y F1 (clase positiva): medias.

Tabla 1: Resumen de los estimadores ajustados y su rendimiento en validación cruzada de 5 particiones (mismas particiones para todos los modelos).

3.1. Comparativa por familias

Todos los modelos lógicos superan la línea base salvo la Greedy Rule List, aunque el margen es moderado (entre +1,6 y +2,8 puntos). Esto es coherente con el análisis exploratorio, donde la señal se concentra en unas pocas variables de comportamiento.

- **CART:** con `min_samples_leaf` = 1 el árbol sobreajusta a partir de profundidad 6 (la exactitud cae hasta 0,792 con profundidad 12), mientras que con `min_samples_leaf` = 200 las curvas se estabilizan. La exactitud es prácticamente plana desde profundidad 4 (0,813–0,815), por lo que el criterio de selección elige profundidad 4 (14 hojas, 81,33 %) frente a la mejor configuración, de profundidad 7 (81,52 %).
- **GOSDT:** con solo 4 hojas ya alcanza 81,32 %, lo mismo que CART con 14, y con 7 hojas obtiene la mejor exactitud del estudio (81,54 % ± 0,20). Su AUC es menor (0,649) porque un árbol de 7 hojas solo produce 7 puntuaciones distintas y el 82 % de los clientes cae en la misma hoja, lo que limita su capacidad de ordenar clientes.
- **Greedy Rule List:** su exactitud coincide con la línea base porque **ninguna regla supera el** 50 % **de dificultad** y todas predicen "no". La primera regla ("en retraso actualmente", 43,8 %) agrupa a clientes con un mes de retraso (36 %, el grupo más numeroso) con otros de tres o más meses (71–86 %). Como una lista solo sigue aprendiendo sobre los clientes que no cumplen la regla, nunca separa a los más arriesgados.
- **Bayesian Rule List:** sí aprende reglas positivas (80,53 %, AUC 0,744), pero necesita 13 reglas ordenadas y es el modelo más inestable entre particiones.
- **RIPPER:** las reglas de 3 condiciones dan más exactitud pero menos AUC y F1 que las de 2, que son más generales y capturan más positivos a costa de más falsos positivos. Sus 8 reglas son muy precisas (58–96 % de dificultad), pero en conjunto solo cubren unos 1 800 clientes (7 %), por lo que muchos positivos quedan en la regla por defecto (F1 0,311).

3.2. Búsqueda extendida de GOSDT

La Tabla 2 muestra las 5 mejores de las 20 configuraciones de la búsqueda extendida.

Binarización	`depth_budget`	`regularization`	Hojas	Accuracy	Std	AUC	F1
base	**4**	**0.0005**	**7.0**	**0.8154**	**0.0020**	**0.649**	**0.376**
base	4	0.0001	8.0	0.8152	0.0019	0.650	0.378
fina	4	0.0001	8.0	0.8151	0.0018	0.650	0.378
base	4	0.0002	7.8	0.8151	0.0018	0.650	0.377
fina	4	0.0002	7.8	0.8151	0.0018	0.650	0.377

Tabla 2: Top 5 de la búsqueda extendida de GOSDT por exactitud en validación cruzada. Hojas: media entre particiones.

La búsqueda no encuentra un modelo mejor. Las 5 mejores configuraciones son árboles de profundidad 4 con 7 u 8 hojas y exactitudes dentro de una desviación típica. Por debajo de $\lambda = 0{,}0005$ solo se añade una hoja que no cambia la exactitud, y con `depth_budget` = 3 siempre se obtiene el mismo árbol de 4 hojas (81,32 %). La **binarización fina no aporta mejora**: GOSDT acaba eligiendo los mismos cortes, y con $\lambda = 0{,}0005$ incluso empeora ligeramente (81,38 % ± 0,41) y se vuelve menos estable entre particiones. La búsqueda está saturada, por lo que lo que limita al modelo es la información de las variables y no sus hiperparámetros.

El *threshold guessing* tampoco mejora a los umbrales elegidos a mano (81,36 % ± 0,41 frente a 81,54 % ± 0,20), pero, como se ve en la Sección 4.1, aprende prácticamente el mismo árbol. Por su parte, la caja negra alcanza 81,64 %, solo 0,1 puntos por encima de GOSDT y dentro de una desviación típica.

4. Interpretación de los modelos

4.1. Interpretación global

4.1.1. Modelo lógico

El modelo seleccionado es el árbol GOSDT con `depth_budget` $= 4$ y $\lambda = 0{,}0005$ sobre la binarización base. Sus 7 hojas se recogen en la Tabla 3.

Rama	Condiciones de la hoja	n	P(dificultad)	Predicción
Edad ≥ 25	ratio de pago $\geq 0{,}03$ y retraso en menos de 6 meses	20 472	0.155	No
	ratio de pago $\geq 0{,}03$ y retraso en los 6 meses	847	0.638	**Sí**
	ratio de pago $< 0{,}03$ y ningún retraso en 6 meses	630	0.424	No
	ratio de pago $< 0{,}03$ y algún retraso en 6 meses	804	0.680	**Sí**
Edad < 25	no está en retraso actualmente	1 625	0.243	No
	en retraso y ratio de pago $\geq 0{,}10$	183	0.404	No
	en retraso y ratio de pago $< 0{,}10$	439	0.688	**Sí**

Tabla 3: Hojas del árbol GOSDT final ajustado sobre todo el conjunto de entrenamiento. "Ratio de pago": `payment_to_bill_ratio_6m`; "retraso actual": `latest_repayment_status` ≥ 1.

La primera división separa a los clientes menores de 25 años, que tienen un riesgo algo mayor. El modelo solo predice dificultad en **tres perfiles**: clientes que han estado en retraso los 6 meses, clientes que apenas pagan (menos del 3 % de la factura) y han tenido algún retraso, y clientes jóvenes en retraso actual que pagan menos del 10 %. Como el resto de hojas predicen "no", el árbol puede reescribirse como una lista de solo tres reglas positivas más una regla por defecto, que es la simplificación natural de árbol a lista [8]. Estas tres reglas cubren a 2 090 clientes (8,4 %).

La penalización por dispersión explica por qué el árbol no crece más. Con $\lambda = 0{,}0005$ y un error normalizado por el número de ejemplos, cada hoja adicional debe corregir al menos unos 12 errores de entrenamiento para compensar. Por eso hojas mezcladas como la de riesgo 0,424 no se dividen: separar mejor esos clientes no compensa la complejidad añadida [8].

Comparación con CART. El árbol CART de profundidad 4 cuenta la misma historia con otras variables. Su raíz es el retraso actual (`latest_repayment_status` $> 0{,}5$), que agrupa los códigos -2, -1 y 0 sin tener que interpretarlos, y su segundo nivel es el ratio de pago (0,11) en ambas ramas. Solo 3 de sus 14 hojas predicen "sí" y varias divisiones (edad o límite de crédito) no cambian la predicción, solo refinan la probabilidad, por lo que podrían podarse sin cambiar ninguna clasificación. `latest_repayment_status` y el ratio de pago acumulan en torno al 80 % de la importancia, y sexo, educación, estado civil y plan de ayuda no se usan. Que dos árboles igual de precisos elijan raíces distintas es un ejemplo del *efecto Rashomon*: al haber variables de retraso muy correlacionadas, la ausencia de una variable en el árbol no implica que no sea informativa [8].

Estabilidad de los umbrales. El árbol aprendido con *threshold guessing* utiliza las mismas divisiones: edad $\leq 24{,}5$, retraso en los 6 meses, retraso actual y ratio de pago $\leq 0{,}025$, que equivale a nuestro corte en 0,03 al estar el ratio redondeado a dos decimales. Sus hojas principales tienen exactamente los mismos tamaños (20 472, 1 625 y 847). Un procedimiento guiado por los datos y aplicado sin ver las particiones de validación llega así a los mismos umbrales que elegimos a partir del análisis exploratorio.

4.1.2. Regresión logística

Reservado para la Fase II.

4.1.3. Explainable Boosting Machine

Reservado para la Fase II.

4.2. Interpretación local

4.2.1. Modelo lógico

Fijamos tres clientes, elegidos al azar con semilla dentro de tres perfiles, que se usarán también en la Fase II. En un árbol, la explicación de una predicción es el camino hasta su hoja, que además permite razonar de forma contrastiva: qué tendría que cambiar para que la predicción fuera distinta [8].

- **CR26-102000 (con dificultad, caso claro).** 23 años, 3 meses de retraso actual, retraso en los 6 meses y ningún pago en el periodo. GOSDT lo clasifica como "sí" por la rama de jóvenes: en retraso y ratio de pago $< 0{,}10$ (68,8 %). La decisión es robusta: si tuviera 25 años o más, seguiría siendo "sí" por la rama de ratio de pago $< 0{,}03$ con retrasos (68,0 %). Los demás modelos también predicen "sí", pero cada uno con un motivo distinto: CART por el retraso en los 6 meses (85 %), la BRL por su regla 3 (2 o más meses de retraso, 87 %) y RIPPER porque se activan tres reglas a la vez, una de ellas basada en el estado civil. Es un buen ejemplo de explicaciones distintas para una misma decisión, y de cómo un conjunto de reglas pierde claridad cuando se activan varias a la vez [8].
- **CR26-129090 (con dificultad, historial limpio)** y **CR26-126238 (sin dificultad).** Ninguno ha tenido retrasos y ambos pagan una parte importante de su factura, así que los dos caen en la misma hoja de GOSDT (riesgo 15,5 %, "no"), y también en la misma hoja o regla del resto de modelos. El primero es un falso negativo: con estas variables es indistinguible de un buen cliente, y para que el modelo lo marcara tendría que acumular retrasos. Este tipo de positivos, sin señales previas, explica la mayor parte de la baja sensibilidad de todos los modelos.

5. Discusión

- **Optimización global frente a voraz.** GOSDT, con 7 hojas, iguala la exactitud de CART con profundidad 7 (44 hojas) y supera ligeramente al CART de 14 hojas (81,54 % frente a 81,33 %), aunque esta diferencia está dentro de una desviación típica. Su ventaja real es la dispersión: un modelo que cabe en una tabla de siete filas. Esto confirma la idea vista en clase de no descartar una representación, el árbol, por las limitaciones de un algoritmo, CART [7].
- **Binarización.** Refinar los umbrales o aprenderlos con *threshold guessing* no mejora el modelo. Sí aumenta la confianza en él, porque distintos procedimientos convergen al mismo árbol.
- **Interpretabilidad frente a exactitud.** La caja negra solo gana 0,1 puntos de exactitud, lo que coincide con la idea de que no siempre existe un compromiso entre ambas [10, 7].

Donde sí gana claramente es en AUC (0,768 frente a 0,649): si el objetivo fuera ordenar clientes por riesgo en lugar de clasificarlos, un árbol de 7 hojas se quedaría corto.

- **Listas y conjuntos de reglas.** La Greedy Rule List muestra el problema de construir la lista de forma voraz, y la BRL el de las listas largas: sus reglas solo se entienden en orden, porque cada una incluye implícitamente la negación de las anteriores [8]. Por ejemplo, su primera regla aparta un grupo de bajo riesgo (18 %) y solo después aparecen las reglas de alto riesgo. En RIPPER, la regla 3 es un caso particular de la regla 4, por lo que podría eliminarse sin cambiar ninguna predicción, y condiciones como el estado civil parecen más un artefacto de la discretización que señal real. Una alternativa para la lista sería CORELS [1], que busca la lista óptima.
- **Limitaciones.** La sensibilidad es baja (F1 de 0,376 en el modelo final), porque los clientes con dificultad sin historial previo de retrasos no se pueden distinguir con estas variables. Además, las asociaciones del plan de ayuda (usado por la BRL) no deben interpretarse causalmente, y la auditoría por sexo queda pendiente.

6. Conclusión

En esta Fase I se han comparado cinco algoritmos lógicos bajo una misma validación cruzada estratificada de 5 particiones. El modelo propuesto es el árbol `GOSDT` (`depth_budget=4`, `regularization=0.0005`), con **7 hojas** y una exactitud media de $\mathbf{81{,}54\,\% \pm 0{,}20}$ (AUC 0,649, F1 0,376), frente al 78,8 % de la línea base y al 81,64 % de una caja negra. Sus decisiones dependen del retraso reciente, del número de meses con retraso, de pagar una fracción muy pequeña de la factura y, en menor medida, de la edad, y se pueden resumir en tres reglas positivas. La búsqueda extendida y el *threshold guessing* indican que este árbol es estable y que el margen de mejora está limitado por la información de los datos más que por el algoritmo.

Para la **Fase II** se entrenarán la regresión logística y el *Explainable Boosting Machine* con las mismas particiones, se explicarán los mismos tres clientes con las tres familias y se realizará la auditoría descriptiva por sexo.

Referencias

[1] Elaine Angelino, Nicholas Larus-Stone, Daniel Alabi, Margo Seltzer, and Cynthia Rudin. Learning certifiably optimal rule lists for categorical data. *Journal of Machine Learning Research*, 18(234):1–78, 2018.

[2] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth, 1984.

[3] William W. Cohen. Fast effective rule induction. In *Proceedings of the 12th International Conference on Machine Learning (ICML)*, pages 115–123, 1995.

[4] Benjamin Letham, Cynthia Rudin, Tyler H. McCormick, and David Madigan. Interpretable classifiers using rules and Bayesian analysis: Building a better stroke prediction model. *The Annals of Applied Statistics*, 9(3):1350–1371, 2015.

[5] Jimmy Lin, Chudi Zhong, Diane Hu, Cynthia Rudin, and Margo Seltzer. Generalized and scalable optimal sparse decision trees. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 6150–6160, 2020.

[6] Hayden McTavish, Chudi Zhong, Reto Achermann, Ilias Karimalis, Jacques Chen, Cynthia Rudin, and Margo Seltzer. Fast sparse decision tree optimization via reference ensembles. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9604–9613, 2022.

[7] B. Mihaljević. Interpretable machine learning. Transparencias de la asignatura Inteligencia Artificial Explicable, ETSIINF, Universidad Politécnica de Madrid, 2026.

[8] B. Mihaljević. Machine learning fundamentals: Logical models. Transparencias de la asignatura Inteligencia Artificial Explicable, ETSIINF, Universidad Politécnica de Madrid, 2026.

[9] Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[10] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

[11] Chandan Singh, Keyan Nasseri, Yan Shuo Tan, Tiffany Tang, and Bin Yu. imodels: a Python package for fitting interpretable models. *Journal of Open Source Software*, 6(61):3192, 2021.

[12] I-Cheng Yeh and Che-hui Lien. The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2):2473–2480, 2009.

Uso de IA generativa

- **Código:** se utilizó un asistente de IA generativa (Claude) para generar y depurar partes del *notebook*, en particular los envoltorios de GOSDT y RIPPER compatibles con scikit-learn, la resolución de dependencias de `gosdt` en Google Colab y las búsquedas de hiperparámetros.
- **Redacción:** también se usó para revisar la redacción y el formato LaTeX del informe y comprobar su coherencia con los resultados del *notebook*.
- **Supervisión:** el equipo ha revisado y ejecutado todo el código, y ha verificado los resultados y las interpretaciones incluidas en este informe.